

Solutionnaire du TP3

Partie A : les notes et le sexe

1. Importer et joindre

```
sexe <- read_tsv("/home/public/tidyverse/grades_sex.tsv")
```

```
grades_sexe <- grades |> left_join(sexe, by = "ID_etu")
nrow(grades)
#> [1] 79
nrow(grades_sexe)                # inchangé : aucune clé en double
#> [1] 79
sexe |> count(ID_etu) |> filter(n > 1)    # confirmation
#> # A tibble: 0 × 2
#> #   i 2 variables: ID_etu <chr>, n <int>
grades |> anti_join(sexe, by = "ID_etu")  # tous les étudiants ont une correspondance
#> # A tibble: 0 × 9
#> #   i 9 variables: ID_etu <chr>, D1 <dbl>, D2 <dbl>, E1 <dbl>, D3 <dbl>,
#> #   D4 <dbl>, E2 <dbl>, Annee <dbl>, Cohorte <chr>
```

2. Moyenne de E2 par année et par sexe

```
grades_sexe |>
  summarise(moyenne_E2 = mean(E2), n = n(), .by = c(Annee, Sex)) |>
  arrange(Annee, Sex)
#> # A tibble: 6 × 4
#>   Annee Sex      moyenne_E2      n
#>   <dbl> <chr>      <dbl> <int>
#> 1  2018 Female      68.3     13
#> 2  2018 Male      65.9     17
#> 3  2019 Female      70.5      9
#> 4  2019 Male      76.4     17
#> 5  2020 Female      86.2      9
#> 6  2020 Male      88.7     14
```

Avec 9 à 17 étudiants par groupe, les écarts observés sont à interpréter avec prudence.

Partie B : les vols au départ de New York

3. Nombre de compagnies

```
nrow(airlines)
#> [1] 16
n_distinct(flights$carrier) # vérification : les compagnies présentes dans flights
#> [1] 16
```

4. Les 3 compagnies les plus en retard

```
flights |>
  left_join(airlines, by = "carrier") |>
  summarise(retard_moyen = mean(arr_delay, na.rm = TRUE), .by = name) |>
  slice_max(retard_moyen, n = 3)
#> # A tibble: 3 × 2
#>   name                                retard_moyen
#>   <chr>                                <dbl>
#> 1 Frontier Airlines Inc.              21.9
#> 2 AirTran Airways Corporation         20.1
#> 3 ExpressJet Airlines Inc.            15.8
```

`na.rm = TRUE` est nécessaire : les vols annulés n'ont pas de retard à l'arrivée (`NA`).

5. Distance et retard par destination

```
flights |>
  left_join(airports, by = join_by(dest == faa)) |>
  summarise(distance_moyenne = mean(distance),
            retard_moyen = mean(arr_delay, na.rm = TRUE),
            n_vols = n(),
            .by = name) |>
  arrange(desc(retard_moyen))
#> # A tibble: 102 × 4
#>   name                                distance_moyenne retard_moyen n_vols
#>   <chr>                                <dbl>         <dbl> <int>
#> 1 Columbia Metropolitan                604.          41.8   116
#> 2 Tulsa Intl                          1215           33.7   315
#> 3 Will Rogers World                    1325           30.6   346
#> 4 Jackson Hole Airport                 1876.          28.1    25
#> 5 Mc Ghee Tyson                       639.          24.1   631
#> 6 Dane Co Rgnl Truax Fld              804.          20.2   572
#> 7 Richmond Intl                       281.          20.1  2454
#> 8 Akron Canton Regional Airport        397           19.7   864
#> 9 Des Moines Intl                     1021.          19.0   569
```

```
#> 10 Gerald R Ford Intl          606.          18.2          765
#> # i 92 more rows
```

Un aéroport avec très peu de vols peut avoir un retard moyen extrême : regarder `n_vols` avant de conclure.

6. Nombre de destinations et clés sans correspondance

```
n_distinct(flights$dest)
#> [1] 105

flights |>
  anti_join(airports, by = join_by(dest == faa)) |>
  count(dest)
#> # A tibble: 4 × 2
#>   dest      n
#>   <chr> <int>
#> 1 BQN      896
#> 2 PSE      365
#> 3 SJU     5819
#> 4 STT      522
```

4 destinations de `flights` (des aéroports de Porto Rico et des Îles Vierges) n'existent pas dans `airports`. Avec `left_join()`, ces vols ont `name = NA` : à la question 5, ils sont regroupés dans une seule ligne `NA`, qui mélange 4 aéroports différents.

Et si... ?

1. `inner_join()` au lieu de `left_join()`

```
nrow(flights)
#> [1] 336776
flights |> inner_join(airports, by = join_by(dest == faa)) |> nrow()
#> [1] 329174
nrow(flights) - nrow(inner_join(flights, airports, by = join_by(dest == faa)))
#> [1] 7602
```

Les vols vers les 4 destinations absentes de `airports` (question 6) disparaissent, **sans message**. `inner_join()` n'est pas une erreur en soi, mais il faut savoir ce qu'on perd : `anti_join()` le dit.

2. Un étudiant en double dans `sexe`

```
sexe_double <- bind_rows(sexe, sexe[1, ])
grades |> left_join(sexe_double, by = "ID_etu") |> nrow()
#> [1] 80
```

80 lignes au lieu de 79, et **aucun message** : la ligne de `Etu_1` est dupliquée. Ses notes compteront deux fois dans les moyennes de son groupe. C'est la vérification « nombre de lignes avant et après » qui le révèle. Pour exiger qu'une clé soit unique dans `y` :

```
grades |> left_join(sexe_double, by = "ID_etu", relationship = "many-to-one")
#> Error in `left_join()` :
#> ! Each row in `x` must match at most 1 row in `y`.
#> i Row 1 of `x` matches multiple rows in `y`.
```

3. Des identifiants en minuscules

```
sexe_min <- sexe |> mutate(ID_etu = tolower(ID_etu))
grades |> left_join(sexe_min, by = "ID_etu") |> count(Sex)
#> # A tibble: 1 x 2
#>   Sex      n
#>   <chr> <int>
#> 1 <NA>     79
```

Aucune erreur : aucune clé ne correspond (`"Etu_1" ≠ "etu_1"`), donc `Sex` vaut `NA` pour les 79 étudiants. `anti_join()` l'aurait signalé immédiatement. On harmonise la clé avant de joindre, par exemple avec `tolower()` des deux côtés.

4. Joindre sans `by`

```
left_join(flights, airports)
#> Error in `left_join()` :
#> ! `by` must be supplied when `x` and `y` have no common variables.
#> i Use `cross_join()` to perform a cross-join.
```

`flights` et `airports` n'ont **aucune colonne de même nom** : `dplyr` ne peut pas deviner la clé et produit une erreur.

```
left_join(flights, airlines) |> select(carrier, name) |> head(3)
#> Joining with `by = join_by(carrier)`
#> # A tibble: 3 x 2
#>   carrier name
#>   <chr>   <chr>
#> 1 UA      United Air Lines Inc.
#> 2 UA      United Air Lines Inc.
#> 3 AA      American Airlines Inc.
```

Ici, la seule colonne commune est `carrier` : la jointure fonctionne, avec un message `Joining with by = join_by(carrier)`. Écrivez toujours `by` : si les deux tableaux avaient une autre colonne commune par hasard (par exemple `name`), la jointure se ferait aussi sur elle, sans erreur.

